

Task 01:

Builder

```
GadgetBuilder.java  MobileGadgetBuilder.java  GadgetDirector.java  BuilderDemo.java

package Demo_Codes.Module02_DST.BuilderMethodDPpack;

public class BuilderDemo {
    public static void main(String[] args) {
        GadgetBuilder builder = new MobileGadgetBuilder();
        GadgetDirector director = new GadgetDirector(builder);
        Gadget mobile = director.constructGadget();

        mobile.showSpecs(); // Output: Mobile Specs: Storage = 512GB, Memory = 16GB
    }
}
```

```
package Demo_Codes.Module02_DST.BuilderMethodDPpack;

public interface Gadget { no usages 1 implementation 5 related problems
    void showSpecs(); no usages 1 implementation 1 related problem
}
```

```
Gadget.java  GadgetBuilder.java  MobileGadgetBuilder.java  GadgetDirector.java

package Demo_Codes.Module02_DST.BuilderMethodDPpack;

public interface GadgetBuilder { 2 usages 1 implementation 4 related problems
    GadgetBuilder buildStorage(int storage); no usages 1 implementation 1 related problem
    GadgetBuilder buildMemory(int memory); no usages 1 implementation 1 related problem
    Gadget build(); no usages 1 implementation 1 related problem
}
```

```

package Demo_Codes.Module02_DST.BuilderMethodDPpack;

public class GadgetDirector { no usages 1 related problem
    private Demo_Codes.Module02_DST.BuilderMethodDPpack.GadgetBuilder builder; 2 usages

    public GadgetDirector(Demo_Codes.Module02_DST.BuilderMethodDPpack.GadgetBuilder builder) { no
        this.builder = builder;
    }

    public Gadget constructGadget() { no usages
        return builder.buildStorage(512).buildMemory(16).build();
    }
}

```

```

package Demo_Codes.Module02_DST.BuilderMethodDPpack;

public class Mobile implements Gadget { no usages 2 related problems
    private int storage; 2 usages
    private int memory; 2 usages

    public void setStorage(int storage) { no usages 1 related problem
        this.storage = storage;
    }

    public void setMemory(int memory) { no usages 1 related problem
        this.memory = memory;
    }

    @Override no usages 1 related problem
    public void showSpecs() {
        System.out.println("Mobile Specs: Storage = " + storage + "GB, Memory = " + memory + "GB")
    }
}

```

```
package Demo_Codes.Module02_DST.BuilderMethodDPpack;

public class MobileGadgetBuilder implements GadgetBuilder { no usages 1 related problem
    private Mobile mobile; 4 usages

    public MobileGadgetBuilder() { no usages 1 related problem
        this.mobile = new Mobile();
    }

    @Override no usages
    public GadgetBuilder buildStorage(int storage) {
        mobile.setStorage(storage);
        return this;
    }

    @Override no usages
    public GadgetBuilder buildMemory(int memory) {
        mobile.setMemory(memory);
        return this;
    }

    @Override no usages 1 related problem
    public Gadget build() {
        return mobile;
    }
}
```

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-jav
Mobile Specs: Storage = 512GB, Memory = 16GB

Process finished with exit code 0
```

Task 02:

Adaptor

```
charger.java | iPhoneRecharger.java  
public interface Charger { no usages 2  
    void charge(); no usages 2 implement  
    void removeCharge(); no usages 2 in  
}
```

```
public class ClientAdapterDpMain {  
    public static void main(String[] args) {  
        Iphone iphone = new Iphone16(new Iphone16Adapter());  
        iphone.onCharge();  
        iphone.offCharge();  
    }  
}
```

```
public interface Iphone { no usages 1 implement  
    void onCharge(); no usages 1 implementation  
    void offCharge(); no usages 1 implementation  
}
```

```
public class Iphone16 implements Iphone { no usages
    ⚡ Charger chargerAdapter; 3 usages

    public Iphone16(Charger chargerAdapter) { no usages
        this.chargerAdapter = chargerAdapter;
    }

    @Override no usages
    public void onCharge() {
        chargerAdapter.charge();
    }

    @Override no usages
    public void offCharge() {
        chargerAdapter.removeCharge();
    }
}
```

```
public class Iphone16Adapter implements Charger { no usages
```

```
    Iphone16Charger iphone16Charger; 3 usages
```

```
    public Iphone16Adapter() { no usages
```

```
    {
        iphone16Charger = new Iphone16Charger();
    }
```

```
    @Override 1 usage
```

```
    public void charge() {
        iphone16Charger.charge();
    }
```

```
    @Override 1 usage
```

```
    public void removeCharge() {
        iphone16Charger.removeCharge();
    }
```

```
}
```

```

public class Iphone16Charger implements Charger {
    no usages

    public Iphone16Charger() {
        no usages
        System.out.println("Iphone16Charger ready.");
    }

    @Override
    no usages
    public void charge() {
        System.out.println("⚡ I am charging iPhone 16");
    }

    @Override
    no usages
    public void removeCharge() {
        System.out.println("❌ iPhone 16 charging stopped");
    }
}

```

```

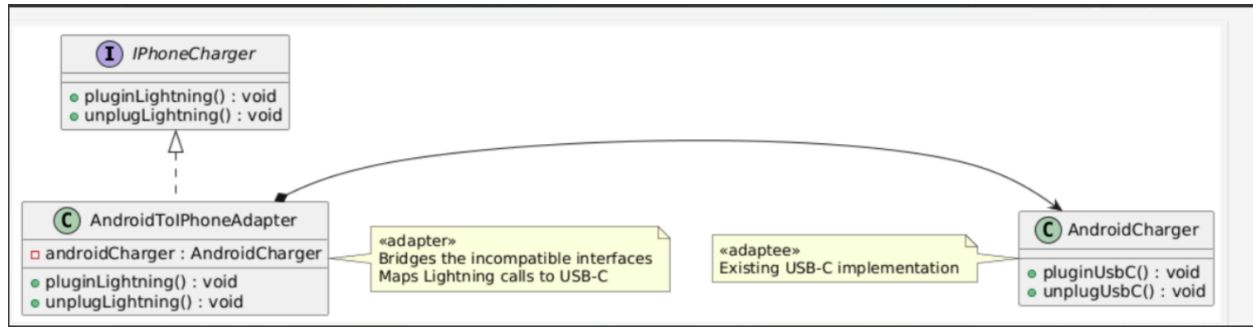
Stop 'ClientAdapterDpMain' Ctrl+F2
⚡ I am charging iPhone 16
❌ iPhone 16 charging stopped

Process finished with exit code 0

```

Task 03

What are the commonly used visibility Notations in class diagrams?



Task4

What are the commonly used visibility Notations in class diagrams?

Symbol	Visibility	Meaning
+	Public	Accessible from any other class.
-	Private	Accessible only within the same class.
#	Protected	Accessible within the class and its subclasses (inheritance).
~	Package	Accessible only within the same package (also called default visibility).

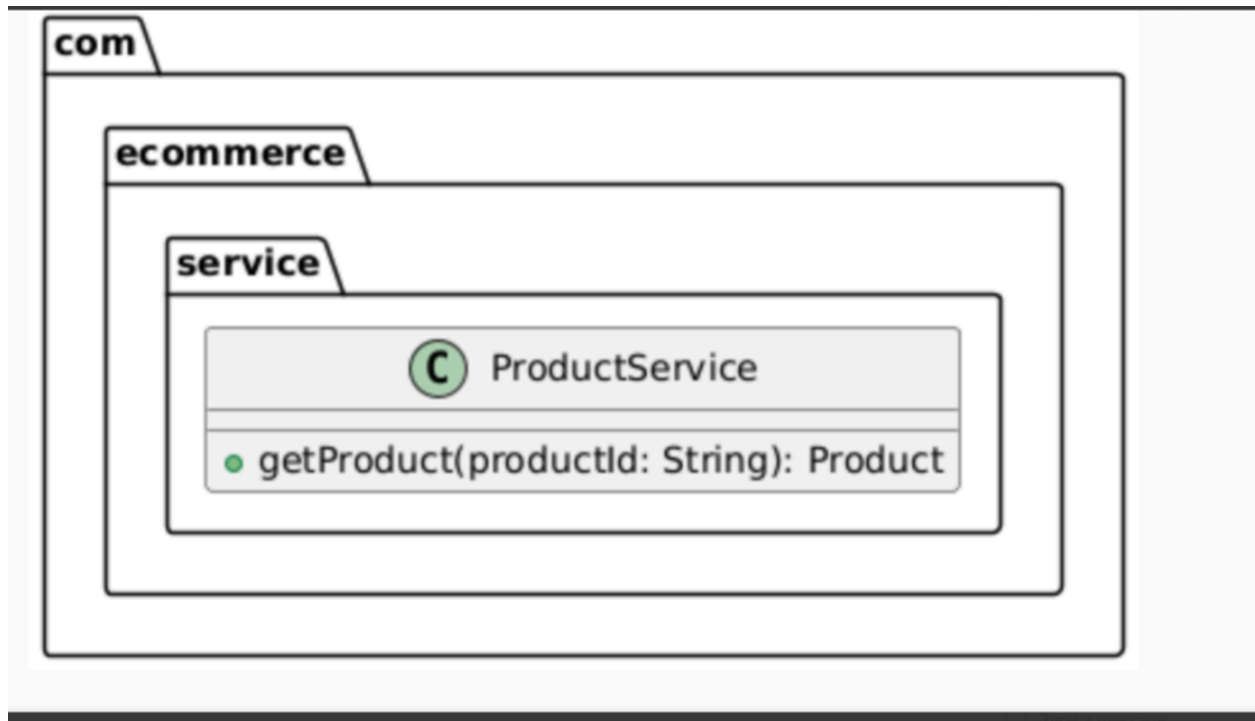
Task 05:

What do you know about Parameter Directionality?

Parameter Directionality refers to the way parameters are used in operations (methods) in **UML (Unified Modeling Language)**, particularly in **class diagrams** and **sequence diagrams**. It defines **how data flows** between objects through parameters when an operation is called.

Direction	Symbol	Meaning
in	(default)	The parameter is used to pass data into the operation (input only).
out	out	The parameter is used to return data from the operation (output only).
inout	inout	The parameter is used to pass data in and out (modified by the operation).

Task 07



@startuml

```
package com.ecommerce.service {
    class ProductService {
        + getProduct(productId: String): Product
    }
}
```

@enduml

Task8

what is state notation .. initial and final ?

In **UML State Diagrams** (also called **State Machine Diagrams**), **state notation** is used to represent the **different conditions or situations (states)** an object can be in during its lifecycle. It helps model **behavioral changes** in response to events.

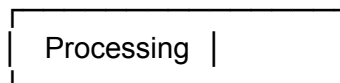
Initial State (Start State):

- **Notation:** A filled black circle (●).

- **Meaning:** The point where the object **begins** its lifecycle.
- **Example:**
- $\longrightarrow$ [StateName]

Final State (End State):

- **Notation:** A circle with a filled black circle inside (⊙).
- **Meaning:** The point where the object **ends** its lifecycle or the process **terminates**.
- **Example:**



Transitions:

- **Arrows** show movement from one state to another.
- Can be triggered by **events** or **conditions**.
- **Example:**
- $\longrightarrow$ Idle $\xrightarrow{[start()]}$ Processing $\xrightarrow{[complete()]}$ ⊙

Homework1

```
Main.java  FacadeDemo.java x
// Service class (subsystem)
class FdService { 2 usages
    public void getFdServiceDetails(String accountNo) { 1 usage
        System.out.println("Fetching FD details for Account No: " + accountNo);
    }
}

// Facade class
class BankFacade { 2 usages
    // list of all services
    private FdService fdService; 2 usages

    // constructor
    public BankFacade() { 1 usage
        this.fdService = new FdService();
    }

    // simplified access method
    public void getFdServiceDetails(String accountNo) { 1 usage
        fdService.getFdServiceDetails(accountNo);
    }
}

// Client code
public class FacadeDemo {
    public static void main(String[] args) {
        BankFacade bankFacade = new BankFacade();
        bankFacade.getFdServiceDetails(accountNo: "ACC12345");
    }
}
```

```
FacadeDemo x
C:\Program Files\Java\jdk-17\bin\java.exe" "-javaa
Fetching FD details for Account No: ACC12345

Process finished with exit code 0
```

Home task 02:

```

// Put imports at the top of the file
import java.util.ArrayList;
import java.util.List;

// Component interface
interface Component { 9 usages 2 implementations
    void brandName(); 2 usages 2 implementations
}

// Leaf class (individual objects)
class Leaf implements Component { 3 usages
    private String brand; 2 usages

    // Constructor
    public Leaf(String brand) { 3 usages
        this.brand = brand;
    }

    @Override 2 usages
    public void brandName() {
        System.out.println("Brand: " + brand);
    }
}

// Composite class (can contain leaves or other composites)
class Composite implements Component { 4 usages
    private List<Component> components = new ArrayList<>(); 3 usages

    public Composite() { 2 usages
    }
}

```

```

    public Composite() { 2 usages
    }

    public Composite(List<Component> components) { no usages
    |     this.components = components;
    }

    @Override 2 usages
    public void brandName() {
    |     for (Component comp : components) {
    |         comp.brandName();
    |     }
    }

    public void add(Component subComponent) { 4 usages
    |     components.add(subComponent);
    }
}

// Client
> public class CompositePatternDemo {
>     public static void main(String[] args) {
    Component dell = new Leaf( brand: "Dell");
    Component hp = new Leaf( brand: "HP");
    Component lenovo = new Leaf( brand: "Lenovo");

```

```
// Client
public class CompositePatternDemo {
    public static void main(String[] args) {
        Component dell = new Leaf(brand: "Dell");
        Component hp = new Leaf(brand: "HP");
        Component lenovo = new Leaf(brand: "Lenovo");

        Composite laptopBrands = new Composite();
        laptopBrands.add(dell);
        laptopBrands.add(hp);

        Composite allBrands = new Composite();
        allBrands.add(laptopBrands);
        allBrands.add(lenovo);

        allBrands.brandName();
    }
}
```

```
C:\Program Files\Java\jdk-17\bin\java.exe
Brand: Dell
Brand: HP
Brand: Lenovo

Process finished with exit code 0
```

Home Task 03:

Proxy:

```
1 import java.util.Objects;
2
3 interface DBExecutor { 3 usages 1 implementation
4     void runQuery(String type); 6 usages 1 implementation
5 }
6
7 class DB { 2 usages
8     DB() { } 1 usage
9
10     void runIt(String type, String id) { 1 usage
11         System.out.println("User: " + id + " | Running query: " + type);
12     }
13 }
14
15 class DBProxy implements DBExecutor { 2 usages
16     String id; 4 usages
17     DB db; 2 usages
18
19     public DBProxy(String id) { 2 usages
20         this.id = id;
21         this.db = new DB();
22     }
23
24     @Override 6 usages
25     public void runQuery(String type) {
26         // Restrict DELETE for non-admin users
27         if (type.equalsIgnoreCase("DELETE") && !Objects.equals(this.id, "ADMIN"))
28             System.out.println("Sorry, access denied for user: " + this.id);
29         return;
30     }
31 }
```

Activate Windows

```

@Override 6 usages
public void runQuery(String type) {
    // Restrict DELETE for non-admin users
    if (type.equalsIgnoreCase("DELETE") && !Objects.equals(this.id, "ADMIN")) {
        System.out.println("Sorry, access denied for user: " + this.id);
        return;
    }
    db.runIt(type, this.id);
}

}

public class ClientProxyDP {
    public static void main(String[] args) {
        DBExecutor obj1 = new DBProxy("ADMIN");
        obj1.runQuery("READ");
        obj1.runQuery("UPDATE");
        obj1.runQuery("DELETE");

        System.out.println("-----");

        DBExecutor obj2 = new DBProxy("MANAGER");
        obj2.runQuery("READ");
        obj2.runQuery("UPDATE");
        obj2.runQuery("DELETE");
    }
}

```

```

C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent
User: ADMIN | Running query: READ
User: ADMIN | Running query: UPDATE
User: ADMIN | Running query: DELETE
-----
User: MANAGER | Running query: READ
User: MANAGER | Running query: UPDATE
Sorry, access denied for user: MANAGER

```

Hmotask 04:

Flyweight


```

1  import java.util.HashMap;
2  import java.util.Map;
3
4  // Flyweight interface
5  interface Laptop {
6      void display(String ram, String storage);
7  }
8
9  // Concrete Flyweight
10 class LaptopModel implements Laptop {
11     private String brand; // intrinsic state (shared)
12
13     public LaptopModel(String brand) {
14         this.brand = brand;
15     }
16
17     @Override
18     public void display(String ram, String storage) {
19         System.out.println("Laptop Brand: " + brand + ", RAM: " + ram + ", Storage: " + storage);
20     }
21 }
22
23 // Flyweight Factory
24 class LaptopFactory {
25     private static final Map<String, Laptop> laptopMap = new HashMap<>();
26
27     public static Laptop getLaptop(String brand) {
28         Laptop laptop = laptopMap.get(brand);
29         if (laptop == null) {
30             laptop = new LaptopModel(brand);

```

Activate Window


```

1 // Implementor
2 interface Color { 6 usages 2 implementations
3     void applyColor(); 2 usages 2 implementations
4 }
5
6 // Concrete Implementors
7 class RedColor implements Color { 1 usage
8     @Override 2 usages
9     public void applyColor() {
10         System.out.println("Applying red color");
11     }
12 }
13
14 class BlueColor implements Color { 1 usage
15     @Override 2 usages
16     public void applyColor() {
17         System.out.println("Applying blue color");
18     }
19 }
20
21 // Abstraction
22 abstract class Shape { 4 usages 2 inheritors
23     protected Color color; // Bridge to implementor 3 usages
24
25     public Shape(Color color) { 2 usages
26         this.color = color;
27     }
28
29     abstract void draw(); 2 usages 2 implementations
30 }

```

```

28
29  abstract void draw(); 2 usages 2 implementations
30  }
31
32  // Refined Abstractions
33  class Circle extends Shape { 1 usage
34      public Circle(Color color) { 1 usage
35          super(color);
36      }
37
38      @Override 2 usages
39      void draw() {
40          System.out.print("Drawing Circle with ");
41          color.applyColor();
42      }
43  }
44
45  class Square extends Shape { 1 usage
46      public Square(Color color) { 1 usage
47          super(color);
48      }
49
50      @Override 2 usages
51      void draw() {
52          System.out.print("Drawing Square with ");
53          color.applyColor();
54      }
55  }
56

```

```

44
45  ✓ class Square extends Shape { 1 usage
46  ✓     public Square(Color color) { 1 usage
47      |         super(color);
48      |     }
49
50      |     @Override 2 usages
51  i↑✓ void draw() {
52      |         System.out.print("Drawing Square with ");
53      |         color.applyColor();
54      |     }
55  }
56
57  // Client
58  ▶✓ public class BridgePatternDemo {
59  ▶✓     public static void main(String[] args) {
60      |         Shape redCircle = new Circle(new RedColor());
61      |         Shape blueSquare = new Square(new BlueColor());
62
63      |         redCircle.draw();
64      |         blueSquare.draw();
65      |     }
66  }
67

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-j
Drawing Circle with Applying red color
Drawing Square with Applying blue color

Process finished with exit code 0

```